

INDIVIDUAL PROJECT REPORT
ON
**“CampusSphere: Database Design, Attendance Verification, Feedback,
Certificate Seal and Activity-Points Engine for a Unified College Event and
Activity-Points Platform”**

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of Technology
in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Anshika Pathak

(Roll No.: _____)

Member, Group Project CampusSphere

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

CERTIFICATE

This is to certify that the individual project report entitled “**CampusSphere: Database Design, Attendance Verification, Feedback, Certificate Seal and Activity-Points Engine for a Unified College Event and Activity-Points Platform**”, submitted by **Anshika Pathak** (Roll No.: _____) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by her as a member of the project group (Govind Jangid, Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak) at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his continuous guidance, encouragement and valuable suggestions throughout the project. His insistence on clear requirements, a weekly plan and honest review helped me understand how a large software system is planned and delivered by a team.

I sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for the academic support and the environment provided for this work. I am also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

I would like to acknowledge the support of my teammates, **Govind Jangid, Garvita Jain, Ayushi Garg and Ankit Yadav**, with whom I worked on **CampusSphere**. Our discussions on interfaces between the frontend, backend and database sections, the joint review of the specification and the shared planning of the 14-week schedule greatly enriched my understanding of team-based software development. I especially thank **Govind Jangid**, our Team Leader, for coordinating the weekly meetings and the integration of all five sections.

I also thank the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events shaped the requirements of the platform.

Finally, I extend my heartfelt gratitude to my family and friends for their constant support, patience and motivation throughout the course of this project.

Anshika Pathak
(Roll No.: _____)
Computer Science & Engineering

ABSTRACT

This report presents my individual contribution to the group project **CampusSphere**, a unified web platform for college events, club treasuries, verified attendance and AICTE activity points at **Arya College of Engineering & I.T., Jaipur**. The work was divided among five students into two frontend and deployment sections (Govind Jangid and Ankit Yadav) and three backend and database sections (Garvita Jain, Ayushi Garg and Anshika Pathak).

My section is the database and the trust-bearing backend modules built on it. It covers the MongoDB schemas and indexes of all collections, the idempotent database seeder, the transaction helper, the gate attendance API with a single-scan guarantee based on a unique index, the attendance-gated five-factor feedback with the suggestion and Kanban APIs, sentiment scoring and suggestion clustering, the certificate serial number with a SHA-256 seal and public verification, the AICTE activity-points engine with an append-only ledger and domain caps, the knowledge data of the Campus Concierge chatbot, and backup, restore and retention.

The report explains the technologies and concepts used, such as document modelling, unique, compound and TTL indexes, multi-document transactions, idempotent jobs, HMAC verification, SHA-256 seals, embeddings and cosine similarity. It describes the design of each module, my 14-week plan, the dependencies on my teammates, the test plan and the challenges anticipated with their planned solutions.

This is a design and planning report. The targets quoted are goals to be measured during testing and the pilot, not measured results. The work provides practical experience in database design, data integrity, concurrency control and secure verification.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	1
1.1 Project Overview	1
1.2 Team and Work Division	1
1.3 My Role and Responsibilities	2
1.4 Objectives of My Section	2
1.5 Project Duration and Schedule	3
1.6 Tools and Collaboration	3
CHAPTER 2 TECHNOLOGIES AND CONCEPTS	4
2.1 Document Modelling in MongoDB	4
2.2 Indexes: Unique, Compound and TTL	4
2.3 Multi-Document Transactions	4
2.4 Idempotent Seeding	4
2.5 HMAC Verification and Constant-Time Comparison	4
2.6 WebSocket Channels	5
2.7 SHA-256 Seals	5
2.8 Sentiment Analysis and Text Clustering	5
2.9 Background Jobs and Exactly-Once Effects	5
2.10 Retrieval-Augmented Generation Data	5
2.11 Backup, Restore and Retention	5
CHAPTER 3 INDUSTRY APPLICATIONS AND RELEVANCE	6
3.1 Document Databases in Production	6
3.2 Check-In and Access Control Systems	6
3.3 Verified Reviews	6
3.4 Tamper-Evident Records	6
3.5 Points and Loyalty Ledgers	6
3.6 Natural Language Processing for Feedback	6
3.7 Future Scope	6
CHAPTER 4 MY WORK: DESIGN AND IMPLEMENTATION APPROACH	7
4.1 Data Layer Structure and Boundaries	7
4.2 Collection Design	7
4.3 Embedding Versus Referencing	8
4.4 Indexes and Integrity Guarantees	9
4.5 Database Seeder	9
4.6 Transaction Helper	10
4.7 Gate Attendance Verification API	10
4.8 Attendance Records and Dispute Override	12
4.9 Feedback Gatekeeper	13
4.10 Suggestions, Upvotes and the Kanban API	13
4.11 Sentiment Scoring and Suggestion Clustering	14
4.12 Certificate Serial, Seal and Public Verification	14
4.13 AICTE Activity Points Engine	15
4.14 Chatbot Knowledge Data	16
4.15 Backup, Restore and Data Retention	16
4.16 Error Codes and Contracts	17
4.17 Testing Approach	17
4.18 Anticipated Challenges and Planned Solutions	18
4.19 Integration with Teammates	18
CHAPTER 5 14-WEEK WORK PLAN	19
5.1 My Week-by-Week Plan	19
5.2 Team Overview and Dependencies	19
5.3 Reporting and Review	20
CHAPTER 6 LEARNING OUTCOMES AND SKILLS	21
6.1 Technical Skills	21
6.2 Professional Skills	21
6.3 Teamwork	21
6.4 Problem Solving and Debugging	22
6.5 Version Control and Communication	22

CHAPTER 7 CONCLUSION	23
REFERENCES	24

23
24

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The group project **CampusSphere** is a unified platform for college events, club treasuries, verified attendance and AICTE activity points at Arya College of Engineering & I.T., Jaipur. It replaces chat-group announcements, form-based registration, personal UPI collection, paper attendance and static certificates with one authenticated system for Students, Club Admins and the Super Admin (Dean/HOD). The platform includes a directory of fifteen clubs with dedicated treasuries, an event approval workflow, venue booking with a 30-minute buffer, atomic seat reservation, Razorpay payments credited to club ledgers, a rolling 30-second QR pass, a camera-based gate scanner, attendance-gated feedback, SHA-256-sealed certificates and an automated activity-points ledger.

The project was carried out by five students over 14 weeks under the guidance of **Er. Ram Babu Buri**. This individual report describes the part assigned to me: the database layer and the trust-bearing backend modules that sit directly on it. These are the schema and index design for all collections, the database seeder, the transaction helper, the gate attendance API with its single-scan guarantee and live counter, the attendance-gated feedback and suggestion modules with the Kanban status API, sentiment scoring and suggestion clustering, the certificate serial number, seal and public verification API, the AICTE activity-points engine, the data for the Campus Concierge chatbot, and backup and restore.

1.2 Team and Work Division

The work was divided into five sections. Two members, Govind Jangid and Ankit Yadav, took the frontend and deployment. Three members, Garvita Jain, Ayushi Garg and Anshika Pathak, took the backend and the database. Table 1.1 shows the division.

Table 1.1: Division of work among the team

Member	Section	Responsibility in brief
Govind Jangid (Team Leader)	Frontend (student experience)	Student screens, rolling QR pass, checkout, certificates and points UI, AI feed, 3D twin; integration lead
Ankit Yadav	Frontend (admin consoles) and deployment	Club and Super Admin consoles, gate scanner PWA, Kanban, Docker, NGINX, CI/CD
Garvita Jain	Backend	Authentication, RBAC, clubs, events, venue buffer engine, audit, security
Ayushi Garg	Backend	Registration, capacity lock, payments, ledger, QR token service, certificate PDF

Member	Section	Responsibility in brief
Anshika Pathak	Database and backend	Schemas, indexes, seeding, transactions, attendance, feedback, seal and points

1.3 My Role and Responsibilities

My role is the **database and backend developer for attendance, feedback, certificate seal and activity points**. I was assigned the following modules:

- the MongoDB schemas of all collections in the entity relationship design, reviewed together with Garvita Jain and Ayushi Garg;
- the unique, compound and time-to-live indexes that enforce the integrity rules of the platform;
- the database seeder for the fifteen clubs, the campus venues, the activity-point rules and the demonstration accounts;
- the transaction helper used for payment settlement (with Ayushi Garg), gate scans and points crediting;
- the attendance records and the POST /attendance/verify-scan API with the single-scan guarantee, the live counter, the manual-entry fallback and the dispute override;
- the feedback gatekeeper, the five-factor ratings with the Verified Attendee badge, and the suggestion, upvote and Kanban status APIs;
- sentiment scoring of reviews and clustering of similar suggestions;
- the certificate serial number, the SHA-256 seal and the public verification API;
- the AICTE activity-points engine with its ledger, domain caps and notifications;
- the knowledge data and retrieval index used by the Campus Concierge chatbot;
- backups, the restore drill, data retention rules and the database documentation.

1.4 Objectives of My Section

- To make the database itself refuse duplicate or inconsistent data, so that correctness does not depend only on application code.
- To guarantee that one ticket can produce at most one attendance record, even when two scanners read it at the same moment.

- To make sure that feedback, certificates and activity points can be obtained only through verified attendance at the venue gate.
- To issue certificate seals that can be recomputed and checked by anyone through a public verification page.
- To credit AICTE activity points exactly once per student per event and to keep every credit explainable.
- To turn free-text feedback into structured, actionable information without exposing the identity of reviewers.
- To make sure that a backup can really be restored and that the restored data passes consistency checks.

1.5 Project Duration and Schedule

The project is planned over 14 weeks and follows the ten-phase roadmap: foundation and authentication (Weeks 1-2), clubs (Week 3), events and venues (Weeks 4-5), registration and rolling QR (Week 6), payments (Week 7), gate scanner (Week 8), feedback (Week 9), certificates and points (Week 10), AI suite (Week 11), 3D twin and deployment (Week 12), integration testing (Week 13) and pilot with documentation (Week 14). My schedule is front-loaded on the database: schemas are ready in Week 2, the seeder in Week 3 and the indexes in Week 4, because every other member works on top of them. The attendance, feedback and points modules follow in Weeks 5 to 10. My week-by-week plan is in Chapter 5.

1.6 Tools and Collaboration

I use MongoDB 7 as a local three-node replica set in Docker (needed for transactions), mongosh and MongoDB Compass for inspection, mongodump and mongorestore for backups, the selected backend stack (Spring Boot 3.3 or Node.js), JUnit or Jest with a test database container, Postman, a small parallel-request script for concurrency tests, Git and GitHub with pull requests, and the team issue board. The team meets weekly and records every change to a shared interface in an interface table.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter explains the technologies and concepts that I use for the database and backend modules assigned to me.

2.1 Document Modelling in MongoDB

MongoDB stores records as documents. The central design question is whether related data should be **embedded** in one document or **referenced** from another collection. Data that is read together and rarely changes alone, such as a student's academic profile or the facilities of a venue, is embedded. Data that grows without limit or is queried on its own, such as registrations, payments and feedback, is referenced. Counters that must change together with a rule check are kept in the same document so that one atomic update can handle both.

2.2 Indexes: Unique, Compound and TTL

An index is an ordered structure that lets the database find documents without scanning the whole collection. A **unique index** also refuses a second document with the same key, and because the database checks it inside the write itself, no race between two requests can bypass it. A **compound index** covers several fields, and its field order matters: equality fields first, then range fields. A **TTL index** deletes a document automatically after a period, which is used here for audit logs and expired refresh tokens.

2.3 Multi-Document Transactions

Single-document updates in MongoDB are atomic, but some operations change several documents that must succeed or fail together. On a replica set, MongoDB supports ACID transactions across documents. The driver's callback API retries a transaction when the server reports a transient error. Transactions are slower than single-document operations and have a default lifetime limit of 60 seconds, so they are kept short, contain no calls to external services and are used only where several documents truly depend on each other.

2.4 Idempotent Seeding

A seeder creates the initial data of a system. A seeder that simply inserts is dangerous because running it twice creates duplicates. An idempotent seeder uses an upsert on a natural key, for example the club slug, and sets the initial values only on insert, so that a second run changes nothing and never overwrites values edited later by an administrator.

2.5 HMAC Verification and Constant-Time Comparison

The rolling QR pass is signed with an HMAC-SHA256 computed from a per-ticket secret seed and the current 30-second window. To verify it, the server recomputes the signature with the stored seed and compares it with the received one. The comparison is done in constant time to avoid timing leaks, and it is combined with a time-window check and a database rule that allows one check-in per ticket.

2.6 WebSocket Channels

A WebSocket keeps a two-way connection open between browser and server. The server can therefore push a message when something happens, without polling. The gate scanner and the club dashboard subscribe to a channel of one event and receive the new check-in count after every successful scan.

2.7 SHA-256 Seals

SHA-256 maps any input to a 256-bit digest, and a one-character change gives a completely different digest. A seal is the digest of the certificate fields together with a private college salt. Anyone can ask the server to recompute the seal from the stored values, but only the college can create a valid one because only the college knows the salt.

2.8 Sentiment Analysis and Text Clustering

Sentiment analysis assigns a label (positive, neutral or negative) and a score to a text. Clustering groups similar texts. Each text is converted into an embedding, which is a vector of numbers, and two texts are similar when the cosine of the angle between their vectors is close to one. Suggestions about the same problem, for example weak Wi-Fi in the hall, then fall into the same cluster and become one actionable item.

2.9 Background Jobs and Exactly-Once Effects

Crediting points for 120 attendees is a batch job that starts when an event is completed. A job can crash halfway or be started twice, so it must be safe to repeat. The standard technique is to make every step idempotent through a unique key: if the same step runs again, the database refuses the duplicate and the job moves on.

2.10 Retrieval-Augmented Generation Data

The Campus Concierge chatbot uses retrieval-augmented generation (RAG). Documents about events, venues and club rules are split into short chunks, each chunk is converted into

an embedding and stored with its source. For a question, the most similar chunks are retrieved and given to the language model, which answers from them.

2.11 Backup, Restore and Retention

A backup is useful only if it can be restored. mongodump writes a compressed copy of the data, and mongorestore loads it into a clean database. A restore drill proves the procedure. Retention rules decide how long personal data is kept, as the Digital Personal Data Protection Act, 2023 requires.

CHAPTER 3

INDUSTRY APPLICATIONS AND RELEVANCE

3.1 Document Databases in Production

E-commerce catalogues, content platforms and mobile back ends widely use document databases because records have flexible fields and are read much more often than written. The hybrid choice of embedding and referencing used here is the standard practice for such systems.

3.2 Check-In and Access Control Systems

Conferences, airports and concert venues scan tickets at the gate and must admit each ticket only once. They depend on a unique record per ticket and on fast validation, which is the same model used by the gate scanner API.

3.3 Verified Reviews

Marketplaces show a "verified purchase" badge so that only real customers can review a product. Tying a review to a gate-verified attendance record applies the same idea to events and removes spam and smear campaigns.

3.4 Tamper-Evident Records

Digital certificates, software releases and land records use hashes and signatures so that an alteration can be detected. The certificate seal is a simple tamper-evident record backed by a server-side secret.

3.5 Points and Loyalty Ledgers

Airlines, banks and retailers keep loyalty points as ledger entries rather than as a single number, so that each balance can be explained and corrected. The activity-points ledger follows the same principle for a student's credits.

3.6 Natural Language Processing for Feedback

Companies analyse thousands of reviews with sentiment scoring and topic clustering to find the problems that matter most. Doing the same for event feedback gives club leads a short list of actions rather than a pile of comments.

3.7 Future Scope

The data layer can be extended with change streams for real-time dashboards, field-level encryption for sensitive fields, sharding by event for very large festivals, a data warehouse for

multi-year analytics, verifiable credential formats such as Open Badges for certificates, automatic transcript submission to the university and anomaly detection on scan patterns.

CHAPTER 4

MY WORK: DESIGN AND IMPLEMENTATION APPROACH

This chapter describes the design of the modules assigned to me. It is based on the project specification suite and on the schedule in Chapter 5.

4.1 Data Layer Structure and Boundaries

My work forms the data layer on which the other four members build. It has two parts. The first is the shared foundation: schemas, indexes, seeder and the transaction helper. The second consists of the modules that I own completely: attendance, feedback, suggestions, certificate seal, points and chatbot knowledge. Every module accesses collections only through its own repository class, and a collection has exactly one owner module that may write to it. For example, only the attendance module writes to `attendance_records`, and only the points module writes to the `activity-points` ledger. Other modules read through services and never write directly.

4.2 Collection Design

Figure 4.1 shows the entity relationships and Table 4.1 lists the collections with the main design decision of each. The hybrid strategy of the architecture is followed: tightly coupled data is embedded, and high-volume or independently queried data is referenced.

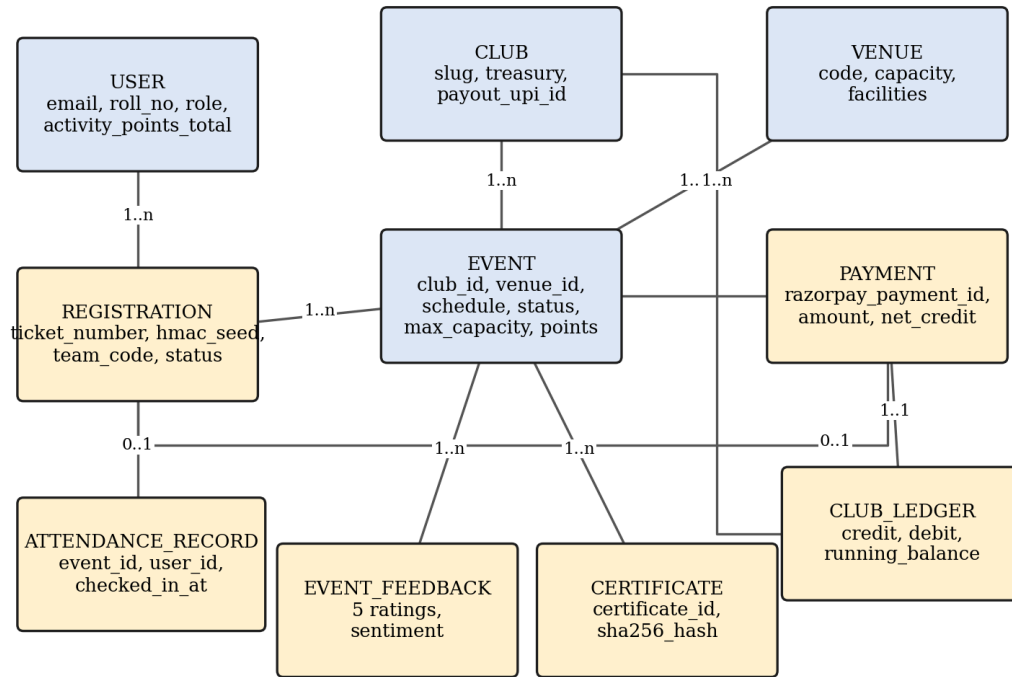


Figure 4.1: Entity relationship overview of CampusSphere

Table 4.1: Principal collections and design decisions

Collection	Main fields	Design decision
users	email, roll_no, role, profile, interests, points_by_domain, activity_points_total	Profile embedded; role never accepted from a client
clubs	slug, category, faculty_coordinator, total_revenue, available_balance, payout_upi_id	Balances change only through ledger transactions
venues	code, name, capacity, facilities, is_active	Facilities embedded as a small array
events	club_id, venue_id, slug, status, schedule, ticketing, category_code, activity_points	Counters kept inside ticketing so one update checks and changes them
registrations	event_id, user_id, ticket_number, hmac_seed, status, attendance_verified	Seed excluded from every default projection
attendance_records	event_id, user_id, registration_id, checked_in_at, method, scanner_id	One record per attendee, enforced by index
event_feedback	event_id, five ratings, overall, review, sentiment, verified	Holds no author field
feedback_authors	event_id, user_id, feedback_id	Separate link, readable only by the service
suggestions	event_id, club_id, text, upvotes, upvoted_by, status, cluster_id	Status drives the Kanban board
suggestion_clusters	event_id, topic, centroid, count	Rebuilt incrementally
certificates	certificate_id, sha256_hash, user_id, event_id, issued_at	Salt is never stored in the database
activity_points_ledger	user_id, event_id, category_code, domain, points, certificate_id, credited_at	Append-only; every row kept even when a cap applies

Collection	Main fields	Design decision
point_rules, domain_caps	category_code, domain, points; domain, cap	Configuration kept as data, not code
audit_logs	user_id, ip_address, action, diff, timestamp	TTL of 365 days
knowledge_chunks	source_type, source_id, text, embedding, updated_at	Public data only; no personal data

The payments, club_ledgers and refresh_tokens collections are designed with Ayushi Garg and Garvita Jain. I review their schemas and indexes so that all collections follow one naming convention, one timestamp format and one error envelope.

4.3 Embedding Versus Referencing

Three rules decide each case. A child that is always read with its parent and has a bounded size is embedded (venue facilities, the ticketing counters of an event). A child that grows with usage or is queried independently is referenced (registrations, attendance, feedback). Data with different access rights is separated even if it is small, which is why the author link of an anonymous review is stored in feedback_authors and not in the review: a Club Admin who can read reviews can never see who wrote them.

4.4 Indexes and Integrity Guarantees

Wherever possible an invariant is enforced by a unique index rather than by application code, because an index cannot be bypassed by a race between two requests. Table 4.2 lists the indexes and the rules they protect. For each compound index I check the explain plan so that the planned query uses it and does not scan the collection.

Table 4.2: Indexes and the invariants they protect

Index	Purpose or invariant
users(email), users(roll_no): unique	One account per identity
clubs(slug), venues(code), events(slug): unique	Stable identifiers and public URLs; makes seeding idempotent
events(venue_id, schedule.start_time, schedule.end_time)	Fast venue buffer collision scan (Garvita Jain)
events(status, schedule.start_time)	Scheduler finds events due to become LIVE or COMPLETED
registrations(event_id, user_id): unique; registrations(ticket_number): unique	No duplicate registration; unique ticket identity
payments(razorpay_payment_id): unique	No double credit of a payment (Ayushi Garg)
attendance_records(event_id, user_id): unique	Single check-in guarantee
attendance_records(event_id, checked_in_at)	Live counter, export and reports
feedback_authors(event_id, user_id): unique	One review per attendee per event
suggestions(event_id, status)	Fast Kanban columns

Index	Purpose or invariant
certificates(certificate_id): unique; certificates(event_id, user_id): unique	Verifiable serial; no second certificate for one attendee
activity_points_ledger(user_id, event_id): unique	No double credit of points
activity_points_ledger(user_id, credited_at)	Transcript listing in date order
club_ledgers(club_id, timestamp desc)	Treasury view of recent entries
audit_logs(timestamp): TTL 31,536,000 s	Automatic deletion after 365 days
refresh_tokens(expires_at): TTL 0	Expired tokens removed automatically

Collection-level validators written as `$jsonSchema` add a second layer. They reject a document with a missing required field, a rating outside 1 to 5 or a status outside the allowed list, even if a bug in the application layer lets it through.

4.5 Database Seeder

The seeder prepares a new environment with the data that the platform needs on the first day. It is idempotent: every record is written with an upsert on its natural key and with `$setOnInsert`, so running it twice gives the same result and never overwrites a value that an administrator has since edited. The demonstration accounts are created only in development and staging, never in production. Table 4.3 lists its contents.

Table 4.3: Contents of the seeder

Data set	Content	Key used for upsert
Clubs	The fifteen master clubs with logo, faculty lead, student coordinator and an empty treasury (zero balances)	slug
Venues	Main auditorium, Seminar Halls A and B, central lawn and laboratories with capacity and facilities	code
Point rules	The eight category codes with their points (Table 4.6)	category_code
Domain caps	Technical Innovation 40, Social and Community 30, Literary and Cultural 20, Sports and Mind Games 10	domain
Demo accounts	One Student, one Club Admin and one Super Admin for testing	email
Sample events	A few events in each state for UI and test work	slug

4.6 Transaction Helper

Three operations in the platform need several documents to change together: confirming a payment, recording a gate scan and crediting points. I provide one helper that all three use, so that retry behaviour and settings are identical everywhere. It wraps the driver's callback API:

```

async function withTxn(fn) {
  const session = client.startSession();
  try {
    return await session.withTransaction(
      () => fn(session),
      { readConcern: { level: 'snapshot' },
        writeConcern: { w: 'majority' } });
  } finally {
    await session.endSession();
  }
}

```

The driver retries the callback when the server reports a transient error, so the callback must not have effects outside the database. For this reason calls to Razorpay, the PDF generator, the language model and the notification service are always made before or after the transaction, never inside it. Table 4.4 lists the three uses.

Table 4.4: Transaction boundaries

Operation	Documents changed together	Reason
Payment confirmation (Ayushi Garg)	registrations, payments, club_ledgers, clubs	Ticket, payment and club credit must exist together or not at all
Gate scan	attendance_records, registrations, events (check-in counter)	Attendance, flag and counter must agree
Points credit for one student	activity_points_ledger, users	A ledger row without a total, or the reverse, would be a silent error

4.7 Gate Attendance Verification API

POST /attendance/verify-scan is called by the scanner of Ankit Yadav. It is allowed for the Club Admin of the organising club (checked against the clubs claim in the token) and for the Super Admin. The request carries the scanned payload, the event selected on the scanner and the method, which is CAMERA or MANUAL. The pipeline of Figure 4.2 runs in this order and stops at the first failure.

- The event must be LIVE and the caller must be authorised for it.
- The Base64URL payload is decoded; a malformed payload gives QR_INVALID.
- The event in the payload must match the selected event, otherwise WRONG_EVENT.
- The window number must be T-1, T or T+1 and the timestamp difference must not exceed 45 seconds, otherwise QR_EXPIRED.
- The registration is loaded by ticket number; it must be CONFIRMED and belong to the user in the payload, otherwise TICKET_NOT_VALID.

- The HMAC is recomputed with the stored seed by the validation library of Ayushi Garg and compared in constant time, otherwise QR_INVALID.
- Inside one transaction the attendance record is inserted, the registration is flagged and the live counter is incremented.

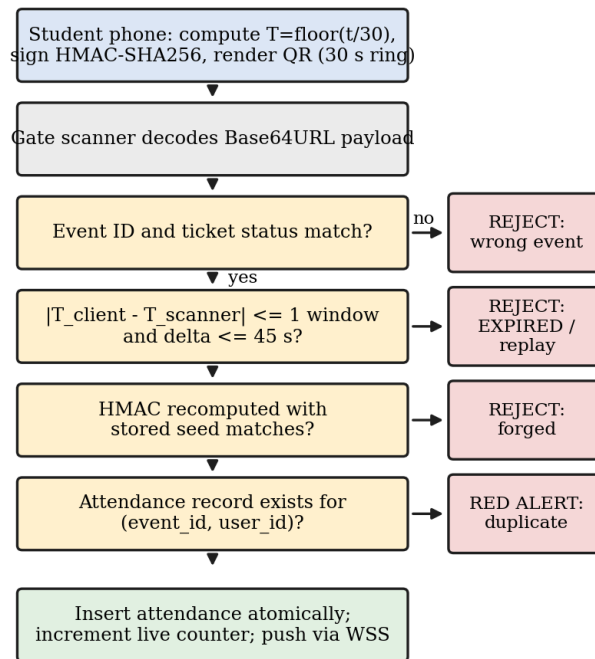


Figure 4.2: Gate-scan validation pipeline

The duplicate check is **not** a read followed by a write, because two scanners reading the same ticket at the same moment would both see no record and both write. Instead the insert is attempted directly and the unique index on (event_id, user_id) decides. The insert that loses receives a duplicate-key error, which the service converts into the red result:

```

try {
  await withTxn(async (s) => {
    await attendance.insertOne(rec, { session: s });
    await regs.updateOne({ _id: reg._id },
      { $set: { attendance_verified: true } }, { session: s });
    await events.updateOne({ _id: ev._id },
      { $inc: { 'ticketing.checked_in_count': 1 } }, { session: s });
  });
} catch (e) {
  if (e.code === 11000) { // duplicate key
    const first = await attendance.findOne({ event_id, user_id });
    throw new DuplicateCheckIn(first.checked_in_at);
  }
  throw e;
}

```

After the commit the service publishes the new count on the event channel, and the scanner and the club dashboard update without polling. A successful response contains the student name, roll number and team name so that the volunteer can compare them with the person at the gate. This matters because lending a live phone cannot be prevented by cryptography and can only be handled by a visual check. Table 4.5 shows the results the scanner can receive.

Table 4.5: Scan results and scanner screens

Result	HTTP	Error code	Scanner screen
Valid first scan	200	none	Green: name, roll number, team
Second scan of the same ticket	409	DUPLICATE_CHECK_IN	Red with alarm and the first check-in time
Old or future token	400	QR_EXPIRED	Red: ask the student to refresh the pass
Signature does not match	400	QR_INVALID	Red
Pass for another event	400	WRONG_EVENT	Red with the event name of the pass
Cancelled or unpaid ticket	403	TICKET_NOT_VALID	Red

Manual fallback. If the camera fails or the venue network is weak, the volunteer can enter the roll number. The same checks on registration status and the same unique index apply, but no signature is available, so the record is stored with method `MANUAL`, the scanner identity and a mandatory reason, and it is listed for review. The comparison of the live counter with the venue capacity and the audit of manual entries are the controls against a careless or dishonest scanner.

4.8 Attendance Records and Dispute Override

An attendance record contains the event, user and registration identifiers, the ticket number, the server time of check-in (never the device time), the method (`CAMERA`, `MANUAL` or `OVERRIDE`) and the identity of the scanner. A student who was present but not scanned can raise a dispute. The Club Admin and the faculty coordinator confirm with evidence, and the override creates a record with method `OVERRIDE` that stores both identities and the evidence note. The action is also written to the audit log. Because points are credited by an idempotent job (Section 4.13), a late override simply makes the student eligible the next time the job runs for that event, and no manual correction of totals is needed.

4.9 Feedback Gatekeeper

`POST /feedback/submit` applies four checks in a fixed order, so that the response always names the first rule that fails:

- the student must hold a registration for the event, otherwise 403 REGISTRATION_REQUIRED;
- the event must be COMPLETED, otherwise 400 EVENT_NOT_COMPLETED;
- a gate-created attendance record must exist, otherwise 403 VERIFIED_ATTENDANCE_REQUIRED;
- no earlier review of this student for this event may exist, otherwise 409 FEEDBACK_ALREADY_SUBMITTED.

Only then are the five ratings (content depth, organisation, speaker quality, venue facilities and value for time), each an integer from 1 to 5, stored together with an overall rating, which is their mean rounded to one decimal, and the review text with a length limit. The last check is not left to a read. In one transaction the service inserts the row into `feedback_authors`, whose unique index on (`event_id`, `user_id`) allows only one review, and then inserts the review itself. The review is displayed with the Verified Attendee badge. Any attempt to bypass these rules through API inspection returns 403 and creates an audit notification.

4.10 Suggestions, Upvotes and the Kanban API

Verified attendees can post public suggestions after the same attendance check. An upvote must count once per student even when the button is pressed twice or two requests arrive together. The rule is placed in the update condition so that the check and the change are one atomic step:

```
const r = await suggestions.updateOne(
  { _id: id, upvoted_by: { $ne: userId } },
  { $push: { upvoted_by: userId }, $inc: { upvotes: 1 } });
if (r.matchedCount === 0) /* already upvoted: no change */;
```

PATCH `/suggestions/:id/status` is allowed for the Club Admin of the event's club and moves a suggestion forward through SUBMITTED, UNDER_REVIEW, PLANNED and IMPLEMENTED. Only the next stage is accepted, and the request carries a note and, for IMPLEMENTED, an optional proof photograph. Every change is stored with the actor and the time, so that the public board of Ankit Yadav can show the feedback loop honestly: what was said and what was done.

4.11 Sentiment Scoring and Suggestion Clustering

Both tasks run in the background so that a student never waits for them. When a review is saved, a job sends only the text (without name or roll number) to the classifier, which returns

a label and a score between -1 and 1; the result is stored in the review. If the service is unavailable, the review is stored with the label PENDING and a retry job fills it later.

Suggestions are clustered incrementally. The text is converted to an embedding vector v and compared with the centroid c of each existing cluster of the same event using cosine similarity:

$$\cos(v, c) = (v \cdot c) / (|v| \times |c|) \quad (1)$$

If the best similarity is at least a threshold θ (for example 0.80), the suggestion joins that cluster, its count is increased and the centroid is updated as a running mean; otherwise a new cluster is created. The topic of a cluster is a short title generated from its first suggestions. A Wi-Fi complaint repeated by many attendees thus becomes one item with a count, which the Club Admin can move on the board. The threshold is a configuration value that I tune on sample data in Week 11.

4.12 Certificate Serial, Seal and Public Verification

After an event is COMPLETED, every verified attendee receives a certificate record. The serial number has the form CS-ARYA-2026-HACK-0142, where the last part is a sequence number per event category and year, taken from an atomic counter document with \$inc so that two certificates never receive the same number. The seal is computed from the certificate fields and the private college salt SALT:

$$Seal = SHA-256(CertificateID | RollNo | EventID | IssueDate | SALT) \quad (2)$$

The fields are joined with a fixed delimiter, so that two different field combinations cannot give the same input string. The salt is read from the secret store of the deployment and is never written to the database or to a log. The record stores the seal in sha256_hash. The certificate module of Ayushi Garg prints the serial, the seal and a QR code of the verification URL on the PDF.

GET /certificates/verify/:certId is public and rate limited. It loads the record, recomputes the seal from the stored values and compares it with the stored hash. It returns the student name and roll number, the event title and organising club, the points, the issue date and the Dean's seal status, and nothing more. Since the page reads values from the database and not from the PDF, a PDF edited in a graphics tool cannot influence the result. An unknown serial gives 404 CERTIFICATE_NOT_FOUND, and a record whose recomputed seal differs is reported

as invalid and logged for investigation. The unique index on (event_id, user_id) makes the generation safe to repeat.

4.13 AICTE Activity Points Engine

The status scheduler of Garvita Jain moves an event to COMPLETED, and this triggers my engine (Figure 4.3). It runs as a background job with these steps:

- claim the job by changing the event's points-job state from NOT_STARTED to RUNNING with one conditional update, so that only one instance works on it;
- read the verified attendees from attendance_records;
- for each attendee, run one short transaction that inserts the ledger row, increments the points of the student's domain and recomputes the capped total;
- after each commit, send the notification, for example "Congratulations! You earned 25 AICTE Activity Points.";
- when all attendees are processed, set the state to DONE.

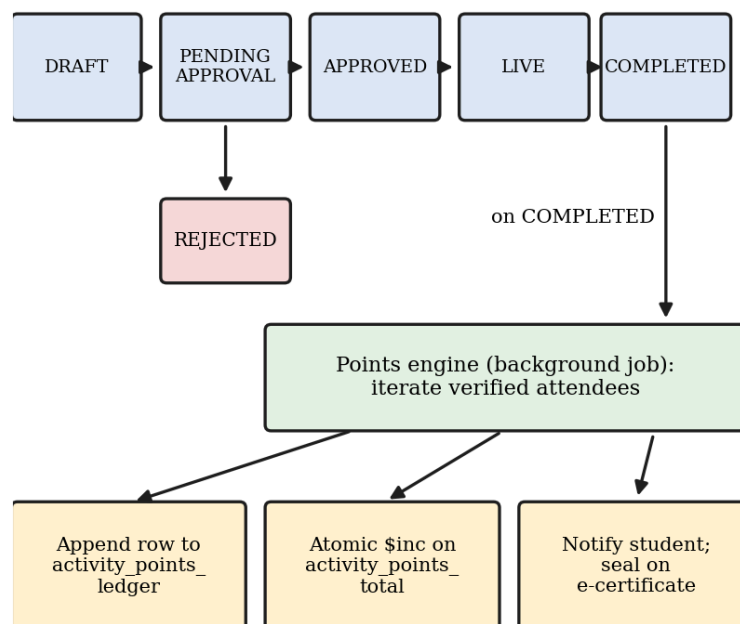


Figure 4.3: Event lifecycle and the points-crediting job

I use one transaction per student, not one for the whole event, because a transaction for 120 or 400 students could exceed the lifetime limit and would roll back everything on one failure. Repeating the job is safe: the unique index on (user_id, event_id) refuses a second ledger row, the service treats that error as "already credited" and moves to the next student, and the

totals are therefore never increased twice. This is the test T10 of the group report: an event with 120 verified attendees must give exactly 120 ledger rows, each total increased by the event points once.

The points and caps follow Table 4.6. Every ledger row is kept, but the total shown to the student is the sum over the four domains of the smaller of the domain total and its cap. The transaction reads the updated domain totals, computes this value and stores it in `activity_points_total`. As the caps are data in `domain_caps`, the Dean's office can revise a cap and a maintenance job recomputes all totals from the ledger without losing any history.

Table 4.6: Category codes, points and domains

Category code	Event types	Points	Domain (cap)
ACT_TECH_HACK	24-36 hour national hackathons	25	Technical Innovation (40)
ACT_TECH_WKSP	Bootcamps and workshops over 8 hours	15	Technical Innovation (40)
ACT_TECH_COMP	Coding contests, robo-wars, FPV racing	10	Technical Innovation (40)
ACT_SOC_CSR	Blood donation, rural tech camps	20	Social and Community (30)
ACT_SOC_ENV	Tree plantation, solar energy expo	15	Social and Community (30)
ACT_LIT_DEB	MUN, parliamentary debate, elocution	15	Literary and Cultural (20)
ACT_CULT_FEST	Inter-college fest, band and dance competitions	15	Literary and Cultural (20)
ACT_SPRT_TOUR	FIDE chess, esports LAN fest	10	Sports and Mind Games (10)

For the official transcript, I provide Garvita Jain with an aggregation that returns the domain totals with caps and the itemised list of events, clubs, dates, gate verification times, certificate identifiers and points. The public view of the transcript uses only the total and the verification status.

4.14 Chatbot Knowledge Data

The Campus Concierge chatbot needs trustworthy data, so I build and maintain its knowledge base. Source texts are the approved and live events, the venues, the club descriptions and the club guidelines. Each source is cut into chunks of a few sentences and stored in `knowledge_chunks` with its embedding, source type, source identifier and update time. Only public information is stored: no names, roll numbers, payments or attendance. A hook on the event approval and update operations refreshes the affected chunks, and a nightly job removes chunks of cancelled events. Retrieval computes the cosine similarity between the question embedding and the chunks, and returns the best few with their sources. For the expected corpus of a few thousand chunks the similarity can be computed in the service, and a vector index can be added if the deployment supports it.

4.15 Backup, Restore and Data Retention

Ankit Yadav schedules the daily backup of the replica set with mongodump into a separate volume and a copy off the server. I write the restore procedure and run the restore drill with him in Week 12: restore into a clean container, then run the consistency checks of Table 4.7 and record the time taken. A backup counts as valid only when these checks pass.

Table 4.7: Consistency checks run nightly and after a restore

Check	Expected result
Available balance of each club equals the sum of its ledger entries	Equal for all fifteen clubs
Every attendance record has a CONFIRMED registration	No orphan record
Check-in counter of an event equals its number of attendance records	Equal
Every points ledger row has a matching attendance record	No orphan row
Total of each student equals the capped sum of the ledger	Equal for all students
Every certificate seal recomputes to the stored hash	No mismatch

Retention follows the data protection design. Audit logs expire after 365 days by the TTL index, expired refresh tokens are removed automatically, and feedback and attendance data have a configurable retention period, after which the author link is deleted and only anonymous aggregates remain. The seeder, the restore script and the checks are kept in the repository so that anyone in the team can run them.

4.16 Error Codes and Contracts

Typical codes of my modules are DUPLICATE_CHECK_IN (409, with the first check-in time), QR_EXPIRED (400), QR_INVALID (400), WRONG_EVENT (400), TICKET_NOT_VALID (403), REGISTRATION_REQUIRED (403), VERIFIED_ATTENDANCE_REQUIRED (403), EVENT_NOT_COMPLETED (400), FEEDBACK_ALREADY_SUBMITTED (409) and CERTIFICATE_NOT_FOUND (404). They use the response envelope of Garvita Jain and are documented in the OpenAPI file, so that the scanner and the student screens can show a precise message.

4.17 Testing Approach

Table 4.8: Test plan for my section

Level	Tool	Examples
Unit	JUnit / Jest	Seal function; capped-total function; window check; rating validation
Integration	Test replica set	A failure inside a transaction rolls back every write
Concurrency	Parallel request script	Two scanners read one ticket together: one 200 and one 409; 50 upvotes by one student count once
Idempotency	Job replay	Points job run twice for 120 attendees gives 120 rows once

Level	Tool	Examples
Index	Explain plans	Venue and status queries use their indexes
Security	Negative tests	Admin of club A scanning for club B gets 403; review without attendance gets 403
Tamper	Edited fixtures	Changed name or points gives a seal mismatch
Restore	Drill	Restored copy passes all checks of Table 4.7

The scan logic depends on time, so the tests use an injectable clock to make the boundary cases around 30 and 45 seconds deterministic.

4.18 Anticipated Challenges and Planned Solutions

Table 4.9: Challenges and solutions

Challenge	Planned solution
Two scanners read one ticket together	Unique index decides; duplicate-key error becomes the red result
Job crash halfway through crediting points	One short transaction per student; unique ledger key makes a rerun safe
Partial writes after a failure	Transactions with the shared helper; no external call inside
Transient transaction errors	Driver retry in the helper; short transactions
Reviewer identity exposed to clubs	Author link in a separate collection with no read endpoint
Weak Wi-Fi at the gate	Manual fallback with reason, scanner identity and later review
Language model unavailable or wrong	PENDING label and retry; no AI text reaches students unreviewed
Changing caps in the future	Caps stored as data; totals recomputed from the full ledger
Backup that cannot be restored	Restore drill and consistency checks in Week 12

4.19 Integration with Teammates

Ayushi Garg and I share the transaction helper and the token validation routine: she writes the HMAC and window rules and I call them in the scan API. Garvita Jain provides the roles and event states that my services check and calls my points data for the transcript. Ankit Yadav builds the scanner, Kanban board, transcript screens and the backup schedule on my APIs, and Govind Jangid builds the feedback, certificate, points and chatbot screens. We agree the payload, the error codes and the field names in Week 6 and record them in the interface table, and I publish sample responses before the real endpoints are ready.

CHAPTER 5

14-WEEK WORK PLAN

5.1 My Week-by-Week Plan

Table 5.1 gives my plan for the 14 weeks together with the expected deliverable of each week. The weeks follow the ten phases of the roadmap.

Table 5.1: My 14-week plan

Week	Phase	My tasks	Deliverable / check
1	Setup	MongoDB replica set in Docker; list of collections from the ERD	Replica set runs; collection list agreed
2	1	Schemas and \$jsonSchema validators for users, clubs, events, registrations	Schemas reviewed with Ayushi Garg and Garvita Jain
3	2	Idempotent seeder: 15 clubs, venues, point rules, caps, demo accounts	Two runs give the same data
4	3	Unique, compound and TTL indexes; explain plans	Duplicates refused; queries use indexes
5	3	Attendance collection, repository and unique index	Second insert for one ticket refused
6	4	WebSocket channel for check-ins; seed storage rules	Seed never returned by any API
7	5	Transaction helper with Ayushi Garg; rollback tests	Failure inside rolls back all writes
8	6	Verify-scan API, duplicate response, manual fallback, override	Valid scan approves; second scan gives 409 with time
9	7	Feedback gatekeeper, ratings, suggestions, upvotes, Kanban status API	Unverified blocked; verified badge shown
10	8	Certificate serial, seal and verify API; points engine and ledger	Tampered record fails; 120 attendees give 120 rows
11	9	Sentiment scoring, suggestion clustering, chatbot knowledge data	Wi-Fi comments in one cluster; relevant chunk retrieved
12	10	Backup and restore drill with Ankit Yadav	Restored copy passes the checks
13	Test	Consistency checks, concurrency and idempotency tests	All invariants hold in CI
14	Pilot	Data dictionary, index list and database runbook	Database documentation delivered

5.2 Team Overview and Dependencies

Figure 5.1 shows the whole team schedule. In every week each member has one main deliverable, and the weekly meeting reviews all five. The dependencies between my tasks and the work of the other members are listed in Table 5.2.

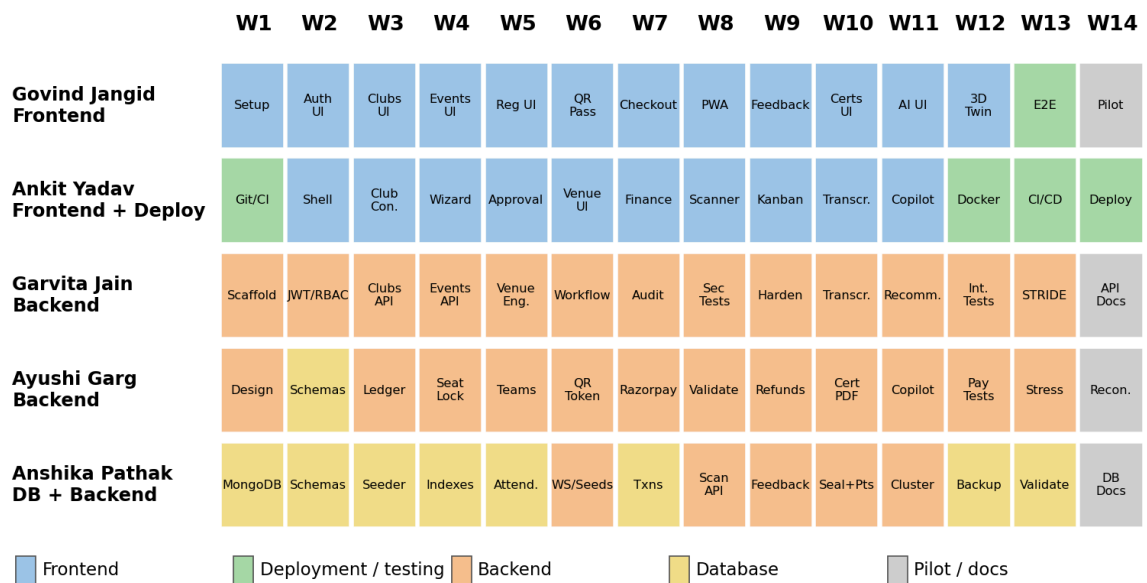


Figure 5.1: 14-week work distribution of the five members

Table 5.2: Dependencies of my weekly tasks on teammates

Week	My task	Depends on / provides to
2	Schemas	Ayushi Garg: registration and payment fields; Garvita Jain: error envelope
3	Seeder	Ankit Yadav and Govind Jangid: club console and club showcase use the seeded clubs
4-5	Indexes and attendance	Garvita Jain: venue query and APPROVED state; Ayushi Garg: registration indexes
6	Seed rules and channel	Ayushi Garg: QR token; Govind Jangid: pass screen; Ankit Yadav: scanner
7	Transaction helper	Ayushi Garg: payment settlement; Ankit Yadav: finance console
8	Verify-scan API	Ayushi Garg: validation library; Ankit Yadav: scanner PWA
9	Feedback and Kanban	Govind Jangid: feedback screens; Ankit Yadav: Kanban board
10	Seal and points	Ayushi Garg: certificate PDF; Garvita Jain: transcript; Govind Jangid: certificates UI
11	Clustering and chatbot data	Govind Jangid: chatbot screen; Garvita Jain: interests and tags for the recommender
12-14	Backup, tests, documents	Ankit Yadav: backup schedule and CI; Garvita Jain: audit entries

5.3 Reporting and Review

Every Friday I give the Team Leader a short status note listing completed tasks, tasks in progress, blockers and changes to interfaces. The guide reviews progress at the end of key milestones (Weeks 2, 5, 7, 9, 11, 13 and 14). A slip is first handled by re-ordering tasks within the week and then, if needed, by moving work between sections in agreement with the

affected members. Because the schemas and indexes are the base of everybody's work, any change to them is announced in the weekly meeting before it is merged.

CHAPTER 6

LEARNING OUTCOMES AND SKILLS

6.1 Technical Skills

- Designing document schemas with a deliberate choice between embedding and referencing.
- Using unique, compound and TTL indexes to enforce integrity rules in the database and checking them with explain plans.
- Writing idempotent seeders and jobs that are safe to run again.
- Using MongoDB transactions with a shared helper, and keeping external calls out of them.
- Verifying HMAC tokens in constant time and designing a single-scan guarantee that holds under concurrent requests.
- Computing and verifying SHA-256 seals for tamper-evident records.
- Designing an append-only points ledger with caps stored as data.
- Applying sentiment scoring and embedding-based clustering to feedback, and preparing retrieval data for a chatbot.
- Planning backups and proving them with restore drills and consistency checks.

6.2 Professional Skills

- Reading and applying a formal specification suite (PRD, SRS, TRD, ERD, API specification).
- Contract-first collaboration between the frontend, backend and database sections.
- Planning work in weekly deliverables and tracking progress with an issue board.
- Writing clear documentation, data dictionaries and status reports.

6.3 Teamwork

The database is the shared base of the project, so a small mistake in a schema or an index reaches all four other members. I learned to review the fields of the registration and payment collections with Ayushi Garg, the event states with Garvita Jain and the response needs of the screens with Govind Jangid and Ankit Yadav before fixing a schema, and to announce every

change early. Working with Ayushi Garg on transactions showed me how one rule, such as "a payment is credited once", must hold in the index, the helper and the tests together.

6.4 Problem Solving and Debugging

Faults in concurrent and data-integrity code are rare but expensive, so I plan to state each rule as an invariant (for example, one attendance record per ticket; one ledger row per student and event; the total equals the capped sum of the ledger), write a test that tries to break it, and change the design if the test succeeds. Reading explain plans and the consistency checks of Table 4.7 gives evidence instead of guesses.

6.5 Version Control and Communication

Short-lived branches, small pull requests and meaningful commit messages make collaboration smoother and keep the main branch always deployable. Schema and index changes are kept in versioned migration scripts so that every environment can be brought to the same state. Clear and timely communication with the other four members, including early notice of changes to an interface, is as important as writing good code.

CHAPTER 7

CONCLUSION

This individual report presented my part in the CampusSphere project: the schemas and indexes of the platform, the seeder and the transaction helper, the gate attendance API, the attendance-gated feedback and suggestion modules, sentiment scoring and clustering, the certificate seal and public verification, the AICTE activity-points engine, the chatbot knowledge data, and backup and restore. The work is divided among five members into two frontend and deployment sections and three backend and database sections, planned over 14 weeks following the ten-phase roadmap.

My modules protect the points where trust is turned into a benefit: a ticket yields at most one attendance record, only a verified attendee can review, receive a certificate or earn points, a certificate can be checked by anyone, and points are credited once and explained by a ledger. The design relies on unique indexes, short transactions and idempotent jobs, which are simple ideas that are easy to test and hard to bypass.

This report documents the design and the plan of my work. The performance and correctness targets are to be verified in Weeks 12 to 14 and in the pilot. The experience has given me a strong base in database design, data integrity, concurrency control and secure verification for professional backend work.

REFERENCES

- [1] AICTE, "Mandatory extracurricular activity points for B.Tech degree," All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota.
- [2] CampusSphere Project Team, "Product Requirements Document (PRD), v1.0.0-PROD," Arya College of Engineering & I.T., Jaipur, 2026.
- [3] CampusSphere Project Team, "Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD," 2026.
- [4] CampusSphere Project Team, "Entity Relationship Diagram and Database Specification (ERD), v1.0.0-PROD," 2026.
- [5] CampusSphere Project Team, "Technical Requirements Document (TRD), v1.0.0-PROD," 2026.
- [6] CampusSphere Project Team, "REST API Specification, v1.0.0-PROD," 2026.
- [7] CampusSphere Project Team, "Security Architecture and Fraud Prevention Specification, v1.0.0-PROD," 2026.
- [8] CampusSphere Project Team, "AICTE / RTU Activity Points Specification, v1.0.0-PROD," 2026.
- [9] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.
- [10] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, May 2011.
- [11] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.
- [12] MongoDB Inc., "Transactions," MongoDB Manual 7.0. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [13] MongoDB Inc., "Indexes," MongoDB Manual 7.0. [Online]. Available: <https://www.mongodb.com/docs/manual/indexes/>
- [14] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [15] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [16] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, Aug. 2023.